Lab U4 - Plug your own planner into Nav2 (Robotics & ROS 2 Essentials, exercise 5)

Aim. The *Robotics & ROS 2 Essentials* repository contains a C++ **Nav2 planner plugin** that does no planning itself: whenever Nav2 needs a path it asks the ROS **service** `/create_plan`. You write that service in Python (here: a straight line), load the plugin into real Nav2 and watch DiffBot follow *your* path.

Time. 60 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload) for the plugin. The Essentials repository is downloaded by `pixi run prepare-essentials`. Source: `upstream\robotics_essentials_ros2` (never modified); a patched copy is built into `upstream_ws\essentials`.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (`winget install --id prefix-dev.pixi -e --source winget`), and Visual Studio Build Tools (C++ workload); internet access once, to download the Essentials repository from GitHub.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Nav2 loads its global planner as a **plugin** (a C++ class loaded by name at start-up). `custom_nav2_planner/CustomPlanner` implements Nav2's planner interface by forwarding the request:

```
Nav2 planner_server --(plugin CustomPlanner)--> service /create_plan
(create_plan_msgs/srv/CreatePlan) --> your node
        start, goal (PoseStamped)  ------------------------------------------------>  returns
nav_msgs/Path
```

`CreatePlan.srv`: request `geometry_msgs/PoseStamped start, goal`; response `nav_msgs/Path path`. Nav2's controller then follows whatever path you return. Nav2 re-plans about once per second, so your service is called repeatedly while the robot drives.

The repository targets ROS 2 Humble. In Jazzy the plugin interface gained a third argument (`cancel_checker`); `scripts\prepare_essentials.ps1` copies the repository's two packages and adds exactly that argument (two lines, see `patches\essentials-jazzy.patch`).

Procedure

1. Prepare and build the repository's two packages (`create_plan_msgs`, `custom_nav2_planner`).

```
pixi run prepare-essentials
pixi run build-essentials
```

2. Read the service you will implement.

```
pixi run cmd /c scripts\in_upstream.bat essentials ros2 interface show
create_plan_msgs/srv/CreatePlan
```

3. T1 - start real Nav2 with the plugin instead of NavFn. `COURSE=1` makes the helper load the course workspace in addition to the repository's overlay. Wait for `Managed nodes are active`; the log must say that the planner is `custom_nav2_planner/CustomPlanner`.

```
$env:COURSE = "1"
pixi run cmd /c scripts\in_upstream.bat essentials ros2 launch diffbot_bringup nav2.launch.py
planner:=custom rviz:=true
```

4. T2 - start your service. Read `code\create_plan_server.py` first: `on_create_plan()` returns a point every 0.1 m on the line start -> goal. It prints one line for every request it gets.

```
$env:COURSE = "1"
pixi run cmd /c scripts\in_upstream.bat essentials python code\create_plan_server.py
```

5. T3 - send a goal. (2.2, 3.0) is free space, 2.3 m from the start, so a straight line is safe. In T2 you see `plan (...) -> (2.20, 3.00)` about once per second, each with fewer points as the robot gets closer.

```
pixi run goto 2.2 3.0
```

In RViz2 the path (`/plan`) is a perfectly straight line, unlike Nav2's own NavFn planner (`planner:=navfn`), which bends the path around the walls.

```
# Ctrl+C in T1 and T2
```

6. The exercise. Replace the body of `on_create_plan()` with a planner that avoids the walls (for example run A* on the map from the `/map` topic) and send `pixi run goto 8.5 6.5`. Nav2 needs no change.

Observations (recorded)

Item	Value
Planner plugin in the Nav2 log	`Created global planner plugin GridBased of type custom_nav2_planner/CustomPlanner`
Service calls during the drive	11 (`Got path as a service result` in the Nav2 log)
Goal (2.2, 3.0)	reached; last feedback 0.35 m, then `REACHED the goal`; about 20 s
Path published on `/plan`	exact straight line from start to goal (deviation under 3 cm)

Verification

```
pixi run test-essentials
```

10 checks: the plugin is loaded and NavFn is not; our service was called at least once; the published plan is the straight line (max deviation < 3 cm); the robot arrives within 0.15 m of (2.2, 3.0), driving about 2.2 m for 2.3 m of straight line, with no collision. Recorded: 10/10.

Questions

1. Why does the plugin return our path unchanged - which Nav2 component then makes sure the robot does not hit a wall?
2. Why is a service (not a topic) the right interface between the plugin and your node?
3. What happens at `goto 8.5 6.5` with the straight-line planner? Try it, and explain what the local costmap and controller do.
4. The repository's plugin source had to be changed in two lines for Jazzy. What does `cancel_checker` allow a planner to do?
5. *(Extension)* Return a path that first goes 1 m sideways (a dog-leg) and verify in RViz2 that Nav2 follows it.

If it does not work

Symptom	Fix
`goto` says "goal rejected"	Nav2 not active yet; wait for `Managed nodes are active`
Nav2 log says `GridBased ... NavfnPlanner`	you started without `planner:=custom`
`No module named create_plan_msgs`	T2 was started without the helper (`scripts\in_upstream.bat essentials`) or without `COURSE=1`
The robot does not move	T2 is not running: the plugin cannot get a path without your service